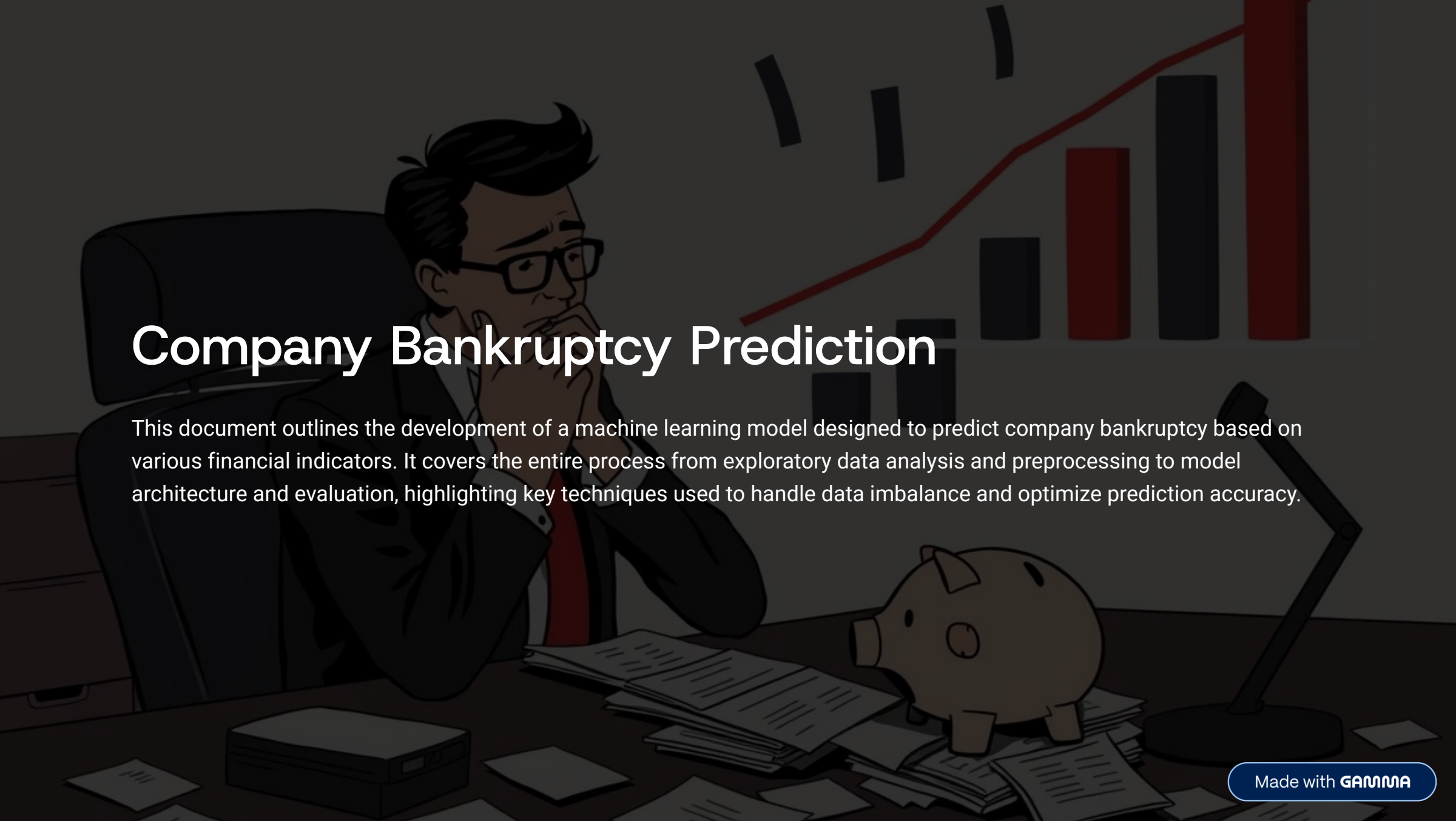


A man in a suit and glasses sits at a desk, looking thoughtful with his hand to his chin. On the desk are stacks of papers, a piggy bank, and a desk lamp. In the background, a bar chart with red and dark blue bars and a red line graph are visible. The overall scene is dimly lit, suggesting a serious or contemplative mood.

Company Bankruptcy Prediction

This document outlines the development of a machine learning model designed to predict company bankruptcy based on various financial indicators. It covers the entire process from exploratory data analysis and preprocessing to model architecture and evaluation, highlighting key techniques used to handle data imbalance and optimize prediction accuracy.

Outline

The development of the bankruptcy prediction model followed a structured approach, encompassing several critical stages:

1 Exploratory Data Analysis

Initial examination of the dataset to understand its structure, identify patterns, and detect anomalies.

2 Data Preprocessing

Preparation of the data for model training, including feature extraction and handling class imbalance through oversampling.

3 Model Architecture

Design and configuration of the machine learning models used for prediction, including a Deep Neural Network and Gaussian Naive Bayes.

4 Evaluation

Assessment of the model's performance using key metrics such as Accuracy, Precision, Recall, and F1 Score.



Exploratory Data Analysis

During the exploratory data analysis phase, several crucial steps were taken to understand the dataset and prepare it for modeling.

Data Overview

- Identified no null values or duplicate rows in the dataset.
- Dataset comprises 7,027 rows and 66 columns (65 features, 1 target variable).

Class Distribution and Imbalance

- Bankrupted companies (Class 1): 271
- Non-Bankrupted companies (Class 0): 6,756
- This highlights a significant class imbalance, which was addressed in later stages.

Feature Correlation

Pairwise correlation was calculated for all feature pairs. If the absolute correlation exceeded a threshold of 0.90, one feature was removed. To decide which feature to keep, the absolute correlation of each feature to the target variable ('class') was considered, retaining the one with higher correlation. This process reduced the number of features from 63 to 50.

Handling Redundant Data and Errors

- Initially, 65 features were identified, with 63 numerical and 2 categorical ('Liability-Assets Flag' and 'Net Income Flag').
- The 'Net Income Flag' was dropped as it had a constant value (1) across all data points.
- Detected and handled features with extremely high mean values due to false data:
- 16 features with over 800 errors were removed.
- 19 features with fewer than 200 errors were corrected using median replacement.
- A total of 50 features were retained for further analysis.



Data Preprocessing: Feature Extraction Using ANOVA

Analysis of Variance (ANOVA) was employed to determine the statistical significance of each feature for model training. This method calculates F-scores for each feature, which indicate how much the feature's values vary between target variable classes compared to within a single class. A high F-score signifies a more significant feature.

$$S_B^2 = \frac{1}{k} \sum_{i=1}^k N_i (\mu_i - \mu)^2$$

Where:

- $k = 2$ (number of classes)
- N_i represents the number of data points in each class
- μ is the mean of all data points for that feature
- $\mu(i)$ is the mean of that feature within a specific class

The variance within groups is calculated by summing the variances in each class, weighted by the sample size of that class. Comparing between-group variance to within-group variance provides a measure of the feature's significance.

```
def select_features_anova(X_data, y_data, top_n=30):  
    selector = SelectKBest(score_func=f_classif, k=top_n)  
    selector.fit(X_data, y_data)  
    mask = selector.get_support()  
    selected_features = X_data.columns[mask].tolist()  
    return selected_features
```

The **SelectKBest** function selects the top 30 features based on their F-scores. The **mask** captures a boolean array indicating selected features, and **selected_features** extracts these chosen features. This process ensures that only the most impactful features are used for training, improving model efficiency and performance.

$$F = \frac{\text{Variance between groups}}{\text{Variance within groups}}$$

Data Preprocessing: Oversampling using SMOTE

To address the significant class imbalance in the dataset, where models tend to be biased towards the majority class, SMOTE (Synthetic Minority Over-sampling Technique) was applied. Initially, the dataset contained 271 samples of the minority class ('1' - bankrupted companies) and 5756 samples of the majority class ('0' - non-bankrupted companies).

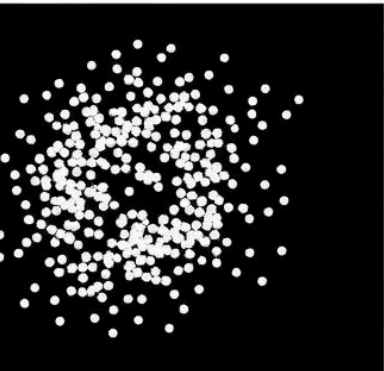
After an 80-20 train-test split, the training data consisted of 5397 samples of class '0' and 224 samples of class '1'. SMOTE works by:

- Randomly selecting a minority class sample.
- Finding its k nearest neighbors.
- Selecting one of these neighbors and generating a new synthetic sample between it and the original sample using linear interpolation.

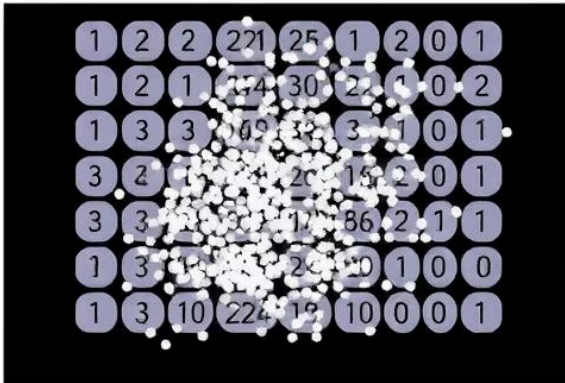
Following the application of SMOTE, the training dataset achieved a balanced distribution, with 5397 samples for each class. It is crucial to note that SMOTE was **not applied to the test data** to ensure an unbiased evaluation of the model's performance on unseen, real-world data.

Before Sataaandia z Attion

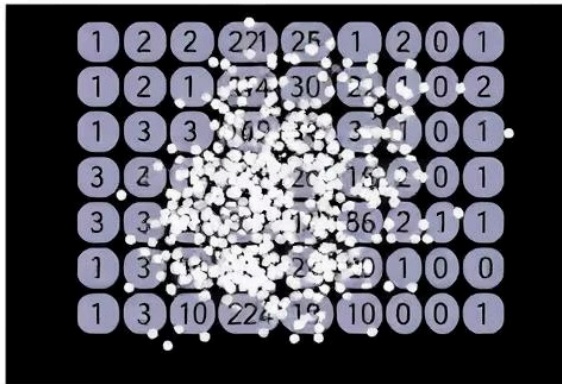
sateripalt:



afteripalt:



ffott mat atclang: 125, 308, 2. If enth mattix, 0 most ctas points
ghtsr, aplany thas the best, clorenth for it the values (0:0)



fott mat atclang: 125, 308, 2. If enth mattix, 0 most ctas points
 ghtsr, aplany thas the best, clorenth for it the values (0:0)

Data Preprocessing: Standardisation

Standardization is a crucial preprocessing step that transforms feature values to have a mean of 0 and a unit variance. This process is essential to prevent features with larger scales from disproportionately influencing the model, ensuring that all features contribute equally to the learning process.

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train_sm)  
X_test_scaled = scaler.transform(X_test)
```

The **StandardScaler** is initialized and then used to fit and transform the training data (**X_train_sm**). The **fit_transform** method calculates the mean and standard deviation from the training data and then applies the transformation. For the test data (**X_test**), only the **transform** method is used. This ensures that the test data is scaled using the same parameters (mean and standard deviation) learned from the training data, preventing data leakage and maintaining consistency between the training and testing phases.

Made with **GAMMA**

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_sm)
X_test_scaled = scaler.transform(X_test)
```

The **StandardScaler** is initialized and then used to fit and transform the training data (**X_train_sm**). The **fit_transform** method calculates the mean and standard deviation from the training data and then applies the transformation. For the test data (**X_test**), only the **transform** method is used. This ensures that the test data is scaled using the same parameters (mean and standard deviation) learned from the training data, preventing data leakage and maintaining consistency between the training and testing phases.

Model Architecture

The bankruptcy prediction model employs an ensemble approach combining a Deep Neural Network (DNN) and a Gaussian Naive Bayes (GNB) model, utilizing soft voting for final predictions.

Deep Neural Network (DNN)

- **Input Layer:** Takes the number of features as input.
- **Hidden Layers:**
 - **First:** 256 neurons, ReLU activation, Batch Normalization, Dropout (0.5).
 - **Second:** 128 neurons, ReLU activation, Batch Normalization, Dropout (0.5).
 - **Third:** 64 neurons, ReLU activation, Batch Normalization, Dropout (0.4).
- **Output Layer:** 1 neuron, sigmoid activation for binary classification.
- **Optimizer:** Adam (learning rate = 0.0005).
- **Loss Function:** Binary Crossentropy.
- **Training:** 200 epochs, batch size 64, 20% validation split.

Gaussian Naive Bayes (GNB)

- Applies Bayes' theorem, assuming feature independence given the class label.
- Calculates bankruptcy probability by multiplying Gaussian likelihoods of each feature and normalizing with prior probabilities.

Ensemble Approach (Soft Voting)

- Predicted probabilities from both models are averaged to obtain the final probability.
- A threshold is tuned based on the F1-score to determine the probability value above which bankruptcy is predicted.

Evaluation

The model was evaluated using a 20% test data split from the "Train.csv" dataset. The performance metrics obtained are as follows:

Precision	0.0128
Recall	0.6667
F1-Score	0.0252
Accuracy	0.9723
Best Threshold	0.4

The high accuracy of 0.9723 indicates that the model correctly predicts a large proportion of both bankrupt and non-bankrupt companies. However, the low precision (0.0128) suggests a high rate of false positives for bankruptcy predictions, meaning many companies predicted to go bankrupt actually do not. Conversely, the high recall (0.6667) indicates that the model is effective at identifying a significant portion of the actual bankrupt companies. The F1-Score (0.0252), which is the harmonic mean of precision and recall, reflects the overall balance between these two metrics, highlighting the challenge of predicting the minority class (bankruptcy) in an imbalanced dataset.